

```
#include <stdio.h>
void display( );
void main( )
{
    void ( *func_ptr )( );

    func_ptr = display; /* assign address of function */
    printf ( "\nAddress of function display is %u", func_ptr );
    ( *func_ptr )( ); /* invokes the function display( ) */
}
void display( )
{
    puts ( "\nLong live viruses!!" );
}
```

The output of the program would be:

```
Address of function display is 1125
Long live viruses!!
```

In **main()**, we declare the function **display()** as a function returning nothing. But what are we to make of the declaration,

```
void ( *func_ptr )( );
```

that comes in the next line? We are obviously declaring something that, like **display()**, will return nothing, but what is it? And why is ***func_ptr** enclosed in parentheses?

If we glance down a few lines in our program, we see the statement,

```
func_ptr = display;
```

so we know that **func_ptr** is being assigned the address of **display()**. Therefore, **func_ptr** must be a pointer to the function **display()**. Thus, all that the declaration

```
void ( *func_ptr )( );
```

means is, that **func_ptr** is a pointer to a function, which returns nothing. And to invoke the function, we are just required to write the statement,

```
( *func_ptr )( ); /* or simply, func_ptr( ); */
```

Pointers to functions are certainly awkward and off-putting. And why use them at all when we can invoke a function in a much simpler manner? What is the possible gain of using this esoteric feature of C? There are two possible uses:

- in implementing callback mechanisms used popularly in Windows programming
- in binding functions dynamically, at run-time in C++ programming

The first of these topics is discussed in Chapter 17 briefly. The second topic is beyond the scope of this book. If you want to explore it further you can refer the book “Let Us C++” or “Test Your C++ Skills” by Yashavant Kanetkar.

Functions Returning Pointers

The way functions return an **int**, a **float**, a **double** or any other data type, it can even return a pointer. However, to make a function return a pointer, it has to be explicitly mentioned in the calling function as well as in the function definition. The following program illustrates this.

```
int *fun( );
void main( )
{
    int *p;
    p = fun ( );
```